



Practice Assignment 6

Student's Full Name:

Student ID:

Instruction:

* Students are allowing to write their answers (like SQL queries, Screen shot of outputs, etc.) in word file (Answer sheet) provided by instructor. After finishing the assignment, students must convert the word file (Answer sheet) into a PDF file. Finally, students upload the file in Moodle.

1. Create the following tables in a new database 'Assignment3':

Clients(**Client Number**, Client_Name, Address, City, Pincode, Province, Amount_Paid, Amount_Due)

Product(**Product Number**, Product_Name, Quantity_On_Hand, Quantity_Sell, Sell_Price, Cost_Price)

Salesman (**Salesman Number**, Salesman _Name, Address, City, Pincode, Province, Salary, Sales_Target, Target_Achieve, Phone)

Salesorder(**Order Number**, Order_Date, **Client Number**, **Salesman Number**, Delivery_Status, Delivery_Date, Order_Status)

Salesorderdetails(**Order Number**, **Product Number**, Order_Quantity)

a) SQL UNION

1. Write a SQL query to find all salesman and clients located in the city of Ha Noi on a table with information: ID, Name, City and Type. And result present below:

	ID	Name	City	Type
▶	S003	Khoa	Thu Dau Mot	Salesman
	S005	Deb	Thu Dau Mot	Salesman
	C102	Le Xuan	Thu Dau Mot	Clients
	C104	Tran Tuan	Thu Dau Mot	Clients

2. Write a SQL query to create a union of two queries that shows the salesman, cities, and target_Achieved of all salesmen. Those with a target of 60 or greater will have the words 'High Achieved', while the others will have the words 'Low Achieved'.
3. Write a SQL query to find those salesman and clients who have placed more than one order. Return ID, name and order by ID.
4. Retrieve the distinct client names who placed orders and the salesman names who processed those orders

b) STORE PROCEDURES

Statements:

1. Create a procedure stores

Delimiter \$\$

CREATE PROCEDURE sp_name () .../

CREATE PROCEDURE sp_name ([IN] param_name type).../

CREATE PROCEDURE sp_name ([OUT] param_name type).../

CREATE PROCEDURE sp_name ([INOUT] param_name type).../

Begin

Select statements;

End\$\$

Delimiter ;

2. Call a procedure stores

Call name_store (value of parameter if have);

3. Drop a procedure stores

Drop procedure name_store;

5. Create a procedure stores get_clients() saves the all Clients in table. Then Call procedure stores.
6. Drop get_clients() procedure stores.
7. Create a stored procedure to retrieve all clients in a specific city.
8. Create a stored procedure to update the delivery status for a given product number.
9. Create a stored procedure to retrieve the total quantity for each product.
10. Create a stored procedure to update the remarks for a specific salesman.
11. Create a procedure stores find_clients() saves all of clients and can call each client by client_number.
12. Create a procedure stores salary_salesman() saves all of clients (salesman_number, salesman_name, salary) having salary >15000. Then execute the first 2 rows and the first 4 rows from the salesman table.
13. Procedure MySQL MAX() function retrieves maximum salary from MAX_SALARY of salary table.
14. Create a procedure stores execute finding amount of order_status by values order status of salesorder table.
15. Create a stored procedure to calculate and update the discount rate for orders.
16. Count the number of salesman with following conditions : SALARY < 20000; SALARY > 20000; SALARY = 20000.
17. Create a stored procedure to retrieve the total sales for a specific salesman.

- c) TRIGGER (when an event happens, do something ex (insert, update, delete) checks data, handles errors, auditing tables)

Syntax:

CREATE TRIGGER trigger_name

{BEFORE | AFTER} {INSERT | UPDATE | DELETE}

ON table_name

FOR EACH ROW

```
BEGIN
    -- Trigger body (SQL statements)
END;
```

```
DROP TRIGGER [IF EXISTS] [schema_name.]trigger_name;
```

18. Create a trigger before_total_quantity_update to update total quantity of product when Quantity_On_Hand and Quantity_sell change values. Then Update total quantity when Product P1004 have Quantity_On_Hand = 30, quantity_sell =35.
19. Create a trigger before_remark_salesman_update to update Percentage of per_remarks in a salesman table (will be stored in PER_MARKS column) : $\text{per_remarks} = \text{target_achieved} * 100 / \text{sales_target}$.
20. Create a trigger before_product_insert to **insert** a product in product table.
21. Create a trigger to update the delivery status to "Delivered" when an order is marked as "Shipped".
22. Create a trigger to update the remarks for all salesmen to "Good" when a new salesman is inserted.
23. Create a trigger to enforce that the first digit of the pin code in the "Clients" table must be 7.
24. Create a trigger to update the city for a specific client to "Unknown" when the client is deleted.
25. Create a trigger to update the delivery status to "Cancelled" for corresponding order details when an order is cancelled.
26. Create a trigger to update the delivery status to "Pending" for a specific order when an order is inserted
27. Create a trigger before_remark_salesman_update to update Percentage of per_remarks in a salesman table (will be stored in PER_MARKS column) If $\text{per_remarks} \geq 75\%$, his remarks should be 'Good'. If $50\% \leq \text{per_remarks} < 75\%$, he is labeled as 'Average'. If $\text{per_remarks} < 50\%$, he is considered 'Poor'.